D5 Errors typical of AI-generated code

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D5 Errors typical of AI-generated code 3
D5-L1 Invented functions, parameters and syntax 3
D5-L2 Deprecated APIs and version mismatches 7
D5-L3 Five review checks for AI-written code 11
Track D best-practice checklist 17
Appendix: solutions and answer keys 18

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D5 Errors typical of AI-generated code

Roadmap: Every month. Invented APIs, deprecated calls, missing edge cases, self-confirming tests.

D5-L1 Invented functions, parameters and syntax

Concept

AI assistants predict plausible code, so they sometimes produce a function, parameter or statement that does not exist, or exists only on another platform. Your reference code has one: `unity_catalog.sql` uses `CREATE ROW ACCESS POLICY`, which is Snowflake syntax. Databricks uses a SQL UDF with `ALTER TABLE ... SET ROW FILTER`.

What invented code does when it runs, checked on PySpark 4.0.4 and pandas 3.0.2:

Invented code	Result
`spark.read.delta(path)`	`AttributeError: 'DataFrameReader' object has no attribute 'delta'`
`df.drop_duplicates(keep="latest")`	`ValueError: keep must be either "first", "last" or False`
`pd.read_csv(path, encodng="utf-8")`	`TypeError: ... unexpected keyword argument 'encodng'. Did you mean 'encoding'?`
`CREATE ROW ACCESS POLICY ...` in Spark SQL	`[PARSE_SYNTAX_ERROR] Syntax error at or near 'ROW'`
`F.col("a").isnotnull()`	`TypeError: 'Column' object is not callable`

The last one is sly. The real method is `isNotNull`. PySpark treats an unknown attribute on a column as a **struct field lookup**, so `F.col("a").isnotnull` builds a valid expression for a field named `isnotnull`. Only the call fails. Without the parentheses, the typo would fail later, at analysis, far from the line that caused it.

Most invented names fail loudly, which makes them cheap to catch **if you run the code**. The expensive cases run: a parameter that exists but means something else, or a function from a newer version than your runtime.

Check every API an assistant gives you:

1. Find it in the official docs **for your version**: Databricks Runtime 17.3 LTS, PySpark 4.0, pandas 3.0.
2. In a notebook, confirm it exists: `hasattr(F, "try_divide")`, `help(pd.DataFrame.merge)`, or `inspect.signature(...)`.
3. For SQL, check the statement in the Databricks SQL reference, not a general SQL tutorial. Syntax from Snowflake, T-SQL and PostgreSQL looks alike and differs.

Treat an unfamiliar name as unverified until a step confirms it.

Why it matters

An assistant proposed `CREATE ROW ACCESS POLICY` to restrict survey rows by region. The engineer pasted it into the migration script without running it. The deployment failed in production at 2 a.m. with a parse error, and the table stayed unrestricted until morning. Checking the statement against the Databricks SQL reference would have taken two minutes.

Worked example: weak code

```
❶ CREATE ROW ACCESS POLICY region_policy
2   AS (region STRING) RETURNS BOOLEAN ->
❷   is_member('analysts_' || region);
4 ALTER TABLE research_silver.nat_respondents ADD ROW ACCESS POLICY
    region_policy ON (region);
```

1. **Line 1.** Snowflake syntax. Databricks fails with PARSE_SYNTAX_ERROR.
2. **Line 3.** is_member checks workspace groups. Unity Catalog guidance favours is_account_group_member.

Worked example: strong code

```
❶ CREATE FUNCTION research_governance.region_filter(region STRING)
2   RETURN is_account_group_member(concat('analysts_', region));
3
4 ALTER TABLE research_silver.nat_respondents
❷   SET ROW FILTER research_governance.region_filter ON (region);
```

1. **Line 1.** A SQL UDF returning a boolean, in a three-level name.
2. **Line 5.** The Databricks statement for row filters (Unity Catalog).

Look up every function, parameter and statement for your exact version, and run it on small data before it reaches a pipeline.

Recap card

- **CREATE ROW ACCESS POLICY** Snowflake syntax. Databricks: SQL UDF plus ALTER TABLE ... SET ROW FILTER.
- **F.col('a').isnotnull()** Unknown attribute becomes a struct field lookup, then 'Column' object is not callable. The method is isNotNull.
- **Three ways to confirm an API exists** Official docs for your version, hasattr or help() in a notebook, inspect.signature for parameters.
- **The costly invented-API case** Code that runs: a real parameter with a different meaning, or a function from a newer version.

Practice

Exercise 1. Multiple choice (level 1)

An assistant writes `spark.read.delta("/Volumes/research/raw/nat")` for Databricks. What happens?

- a) AttributeError: DataFrameReader has no attribute 'delta'
- b) It reads the Delta table
- c) It reads the files as Parquet
- d) A warning, then the read succeeds

Why?

- r1) The reader has format('delta').load(path) and table(name), but no delta() method.
- r2) Delta tables cannot be read by path.

Exercise 2. Predict the output (level 2)

What is printed? (PySpark 4.0)

```
from pyspark.sql import functions as F

expr = F.col("region").isnotnull
print(type(expr).__name__)
```

Exercise 3. Spot the bug (level 2)

An assistant wrote this Databricks SQL to mask the postcode column for non-analysts. Tap the line that does not exist in Databricks, then rewrite it with the Databricks statement.

```
CREATE FUNCTION research_governance.mask_postcode(p STRING)
  RETURN CASE WHEN is_account_group_member('analysts') THEN p ELSE '***'
   END;
```

```
3 CREATE MASKING POLICY postcode_mask ON research_silver.nat_respondents
    (postcode);
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write `unknown_params(func: Callable[..., object], params: list[str]) -> list[str] | None`. Use `inspect.signature` to return the names in `params` that `func` does not accept, in the order given. If `func` accepts `**kwargs`, return `None`: you cannot tell from the signature alone.

What you need to know.

`inspect.signature(func).parameters` maps each parameter name to a `Parameter` object. Test a name with `in`. Each parameter's `.kind` says how it is passed. `inspect.Parameter.VAR_KEYWORD` is the kind of a `**kwargs` parameter, which accepts any keyword name.

```
1 import inspect
2
3 def f(a, b=1, **options): ...
4
5 params = inspect.signature(f).parameters
6 "a" in params                                                    # True
7 params["options"].kind is inspect.Parameter.VAR_KEYWORD         # True
```

Your code must pass these checks:

- Return the names the function does not accept, in order. Keyword-only parameters count as accepted.
- An empty list when every name exists.
- With **kwargs, any name is accepted by the signature. Return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Parameter names that a function does not accept.
2 def unknown_params(
3     func: Callable[..., object],
4     params: list[str],
5 ) -> list[str] | None: ...
```

Starter code:

```
1 def unknown_params(func, params):
2     return []
```

Exercise 5. Architecture design (level 4)

Set the team rule for APIs suggested by an assistant.

Options: Link the official docs page for our version in the pull request, Accept it if the assistant is confident.

1. Before code with a new function or parameter is merged
2. SQL statements such as row filters and masks
3. A function you cannot find in the docs

Sources

- PySpark 4.0.4: spark.read.delta and DataFrameWriter.saveAsDelta raise AttributeError; F.col('a').isnotnull is a Column and calling it raises TypeError; CREATE ROW ACCESS POLICY raises PARSE_SYNTAX_ERROR. (verified: Local run on PySpark 4.0.4, September 2026.)
- pandas 3.0.2: drop_duplicates(keep='latest') raises ValueError; read_csv(encodng=...) raises TypeError with a 'Did you mean' hint. (verified: Local run on pandas 3.0.2 with CPython 3.13, September 2026.)
- Databricks row filters use a SQL UDF and ALTER TABLE ... SET ROW FILTER; column masks use ALTER TABLE ... ALTER COLUMN ... SET MASK. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply (checked 2026-09-25))

D5-L2 Deprecated APIs and version mismatches

Concept

An assistant learned from years of code written for older versions. It often suggests an API that worked in 2021 and has since been deprecated, renamed or removed. Your reference `observability.py` uses `datetime.utcnow()`, deprecated from Python 3.12.

Examples that matter for this course, checked in September 2026:

Old code	Current state	Use instead
`datetime.utcnow()`	Deprecated, Python 3.12	`datetime.now(timezone.utc)`
`df.append(other)`	Removed from pandas (gone in 3.0)	`pd.concat([df, other])`
`df.applymap(f)`	Removed in pandas 3.0	`df.map(f)`
`df["a"][0] = x`	Never updates `df` under pandas 3 Copy-on-Write	`df.loc[0, "a"] = x`
`import distutils`	Removed in Python 3.12	`packaging`, `setuptools`
`input_file_name()`		`_metadata.file_path`

Old code	Current state	Use instead
	Not supported from Databricks Runtime 17.3 LTS	
`mssparkutils` in Fabric	Being retired	`notebookutils`

The chained assignment row is the dangerous one. pandas 3.0.2 emits a `ChainedAssignmentError` warning and leaves the DataFrame unchanged. If warnings are hidden, the update simply does not happen.

Version mismatches cause the opposite problem: code for a **newer** version than yours. A function added in PySpark 4.1 fails on Databricks Runtime 17.3 LTS, which ships Spark 4.0. Pyodide in this app runs pandas 3.0.2. Your laptop might run 3.0.6.

Defend in three ways:

- **State the versions** in every prompt, and in the project: `VERSIONS.md`, `pyproject.toml` pins, the runtime in the job definition.
- **Show warnings** in tests. Run pytest with `-W error::DeprecationWarning` for your own code, so a deprecated call fails the build.
- **Compare versions as numbers**, never as strings. `"3.10" < "3.9"` is `True` as text.

Why it matters

An assistant fixed a data cleaning function by adding `df["weight"][mask] = 1.0` to cap weights. Tests passed on a developer laptop with pandas 1.5. In the pandas 3 environment, the cap never happened. Weights above 5 reached gold unchanged for four waves. The warning had been filtered out in the job's logging configuration.

Worked example: weak code

```
1 import pandas as pd
2
3 def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
4     """Cap weights at cap."""
1     df["final_weight"][df["final_weight"] > cap] = cap
2     return df.applymap(lambda v: v)
```

1. **Line 5.** Chained assignment. Under pandas 3 Copy-on-Write it never changes df.
2. **Line 6.** applymap no longer exists in pandas 3.0: AttributeError.

Worked example: strong code

```
1 import pandas as pd
2
3 def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
1     """Return a copy with final_weight capped at cap. Written for pandas
      3.0."""
5     out = df.copy()
2     out.loc[out["final_weight"] > cap, "final_weight"] = cap
7     return out
```

1. **Line 4.** The docstring states the target version.
2. **Line 6.** One .loc call: row mask and column together. Works in every pandas version.

Assume suggested code targets some other version. Check deprecations for yours, turn warnings into test failures, and compare versions numerically.

Recap card

- **Chained assignment in pandas 3** df['a'][i] = x never updates df (Copy-on-Write). Use df.loc[i, 'a'] = x.
- **pandas 3 removals** DataFrame.append (use pd.concat) and applymap (use DataFrame.map).
- **input_file_name() on DBR 17.3 LTS** Not supported. Use _metadata.file_path.
- **Comparing version strings** Never as text: '3.10' < '3.9'. Compare integer tuples or use packaging.version.

Practice

Exercise 1. Multiple choice (level 1)

An assistant adds `F.input_file_name()` to your bronze job on Databricks Runtime 17.3 LTS. What do you use instead?

- a) The _metadata.file_path column
- b) Keep input_file_name(), it is fine
- c) os.path.basename on the DataFrame
- d) A Python UDF reading the path

Why?

- r1) Databricks release notes state input_file_name is no longer supported from 17.3 LTS. The file metadata column replaces it.
- r2) input_file_name was renamed file_path.

Exercise 2. Predict the output (level 2)

What does this print with pandas 3.0?

```
import pandas as pd

df = pd.DataFrame({"w": [1.0, 9.0]})
df["w"][1] = 5.0
print(df["w"].tolist())
```

Exercise 3. Fill the gap (level 2)

Update this code from pandas 1.x to pandas 3.0.

```
combined = pd._____(1)([waves, new_wave], ignore_index=True)
labels = combined._____(2)(str.strip)
```

Exercise 4. Write the function (level 3)

Write `version_at_least(installed: str, required: str) -> bool` comparing dotted versions such as `"3.0.2"` and `"3.10"` numerically. Ignore anything after the numeric part of each segment (`"4.0.4rc1"` counts as 4.0.4). Missing segments count as 0, so `"3.0"` equals `"3.0.0"`. Raise `ValueError` if a segment has no leading digits.

Your code must pass these checks:

- 3.10 is newer than 3.9. Compare numbers, not strings.
- Pad missing segments with 0.
- Ignore suffixes after the digits in each segment.
- A segment without digits is invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
SEGMENT = ...

# Numeric segments of a dotted version string.
def _parse(version: str) -> tuple[int, ...]: ...

# True if installed is the same as or newer than required.
def version_at_least(installed: str, required: str) -> bool: ...
```

Starter code:

```
def version_at_least(installed, required):
    return installed >= required
```

Exercise 5. Spot the bug (level 3)

This function caps weights, but under pandas 3 the weights come back unchanged. Tap the line, then fix it without changing the caller's DataFrame.

```python
import pandas as pd

def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
    """Return a copy with final_weight capped at cap."""
    out = df.copy()
    out["final_weight"][out["final_weight"] > cap] = cap
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- pandas 3.0.2: no DataFrame.append or applymap; DataFrame.map exists; default string dtype 'str'; chained assignment emits ChainedAssignmentError and leaves the frame unchanged. Python 3.13: distutils missing; utcnow deprecated. (verified: Local run on pandas 3.0.2 with CPython 3.13.12, September 2026.)
- input_file_name is no longer supported from Databricks Runtime 17.3 LTS. (verified: https://docs.databricks.com/aws/en/release-notes/runtime/17.3lts (checked 2026-09-25))
- mssparkutils is renamed NotebookUtils and will be retired. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebook-utilities (see lesson C5-L1))

D5-L3 Five review checks for AI-written code

Concept

Code from an assistant usually runs. The problems hide in what it leaves out. Review it with five checks.

1. Missing edge cases. Assistants write the happy path. Ask what happens with an empty DataFrame, a zero total, a null, a duplicate key, a single row. Your reference `check_weight_distribution` fails on an empty wave for exactly this reason (lesson D4-L3).

2. Self-confirming tests. A test that computes its expected value with the same formula as the code proves nothing:

```python
def test_share():
    assert share(3, 4) == 3 / 4   # repeats the code
```

If the formula is wrong, the test is wrong the same way. Write expected values by hand, from small data you understand: `assert share(3, 4) == 0.75`, `assert share(0, 0) is None`.

3. Secrets in code. Assistants fill in placeholders that look like real tokens, or keep the one you pasted into the prompt. A token in a notebook ends up in Git history. Read secrets at run time: `dbutils.secrets.get(scope, key)` on Databricks, `notebookutils.credentials.getSecret(vault_url, name)` in Fabric, or an environment variable on your laptop.

4. Over-engineering. A 12-line task arrives as an abstract base class, a factory and a config object. Every extra layer is code to test and review. Ask for the simplest version that meets the requirement, then add structure when a second use case appears.

5. The wrong filter context in DAX.
`CALCULATE(SUM(fact_reach[weighted_reach]), ALL(fact_reach))` removes **every** filter on the fact table, including wave and region slicers. A "share of total" measure then divides by the all-time, all-region total. Remove only the filter the measure means to ignore: `ALL(dim_brand)` for a share across brands.

Each check takes a minute. Each catches a class of bug that runs without error.

Why it matters

An assistant wrote a reach function and its tests in one reply. All 12 tests passed. Each test computed its expected value with the same expression as the function, including the same missing `is None` check. A reviewer replaced three expected values with hand-worked numbers, and two tests failed at once: the function returned 0.0 where it should have returned None.

Worked example: weak code

```
1  API_TOKEN = "paste-your-token-here-EXAMPLE"
2
3  def share(part: float, total: float) -> float:
2      return part / total
5
6  def test_share():
3      assert share(3, 4) == 3 / 4
```

1. **Line 1.** A secret in code. The real token replaces the placeholder and lands in Git.
2. **Line 4.** No guard for a zero total.
3. **Line 7.** The expected value repeats the code. It cannot catch a wrong formula.

Worked example: strong code

```
1  import os
```

```
 2
 3 def share(part: float, total: float) -> float | None:
 4     """part / total, or None when total is 0."""
 5     return None if total == 0 else part / total
 6
 7 def api_token() -> str:
 8     """Read the token at run time, never from source code."""
 9     return os.environ["SURVEY_API_TOKEN"]
10
11 def test_share():
12     assert share(3, 4) == 0.75
13     assert share(0, 0) is None
```

1. **Line 5.** The edge case is handled and documented.
2. **Line 9.** The secret stays outside the code. On Databricks use dbutils.secrets.get.
3. **Line 13.** Hand-written expected values, including the edge case.

Review AI code for edge cases, self-confirming tests, secrets, needless structure and, in DAX, filters removed too widely.

Recap card

- **Five review checks for AI code** Edge cases, self-confirming tests, secrets, over-engineering, DAX filter context.
- **Self-confirming test** Expected value computed with the code's own formula. Use hand-worked values instead.
- **Where secrets live** Secret scopes or Key Vault, read at run time. Never in code or prompts.
- **ALL(fact_table) in a share measure** Removes every filter, including wave and region slicers. Use ALL on the one dimension you mean.

Practice

Exercise 1. Multiple choice (level 1)

An assistant writes `assert weighted_pct(r, b) == r / b * 100` as the test for `weighted_pct`. What is wrong with it?

- a) It repeats the implementation, so a wrong formula passes
- b) It should use pytest.approx
- c) Tests must not use multiplication
- d) Nothing: it checks the output

Why?

- r1) A test's expected value must come from outside the code: small data worked by hand.
- r2) Tests should never call the function they test.

Exercise 2. Spot the bug (level 2)

An assistant produced this Databricks helper. Tap the line that must never be committed, then fix it to read the value from the secret scope `survey-api`, key `token`.

```
1 import requests
2
3 API_TOKEN = "tok_EXAMPLE_do_not_use_0000"
4
5 def fetch_wave(wave_id: str) -> dict[str, object]:
6     """Fetch wave metadata from the survey platform."""
7     headers = {"Authorization": f"Bearer {API_TOKEN}"}
8     return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Multiple choice (level 2)

A report has a wave slicer set to W14. An assistant's measure is `Share = DIVIDE(SUM(fact_reach[weighted_reach]), CALCULATE(SUM(fact_reach[weighted_reach]), ALL(fact_reach)))`. What is the denominator?

- a) Weighted reach for all waves, all regions and all brands
- b) Weighted reach for W14 across all brands
- c) Weighted reach for W14 for the current brand
- d) An error, because ALL needs a column

Why?

- r1) ALL(fact_reach) removes every filter on the fact table, including the wave slicer.
- r2) CALCULATE always ignores slicers.

Exercise 4. Write the function (level 3)

An assistant wrote `weighted_mean(values, weights)` as `sum(v * w for v, w in zip(values, weights)) / sum(weights)`. Rewrite it with the edge cases it missed. Return `None` if the lists are empty or the weight total is 0. Skip pairs

where the value or the weight is `None`. Raise `ValueError` if the lists differ in length or any weight is negative.

Your code must pass these checks:

- (10 x 1 + 20 x 3) / 4 = 17.5.
- Empty lists and a zero weight total return None.
- Skip pairs with a None value or weight.
- zip silently truncates different lengths. Raise ValueError, and for negative weights too.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weighted mean that skips missing pairs.
def weighted_mean(
    values: list[float | None],
    weights: list[float | None],
) -> float | None: ...
```

Starter code:

```python
def weighted_mean(values, weights):
    return sum(v * w for v, w in zip(values, weights)) / sum(weights)
```

Exercise 5. Refactor (level 4)

An assistant answered "flag weights above a cap" with the code below. Replace it with one function, `flag_high_weights(weights: list[float], cap: float) -> list[int]`, returning the indexes of weights above `cap`. No classes.

Your code must pass these checks:

- Return indexes of weights strictly above the cap.
- No weights, no flags.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Indexes of weights strictly above cap.
def flag_high_weights(
    weights: list[float],
    cap: float,
) -> list[int]: ...
```

Starter code:

```python
from abc import ABC, abstractmethod


class WeightRule(ABC):
    @abstractmethod
    def check(self, weight):
```

```
        ...


class CapRule(WeightRule):
    def __init__(self, cap):
        self.cap = cap

    def check(self, weight):
        return weight > self.cap


class RuleFactory:
    @staticmethod
    def create(kind, **kwargs):
        if kind == "cap":
            return CapRule(**kwargs)
        raise ValueError(kind)


def flag_high_weights(weights, cap):
    rule = RuleFactory.create("cap", cap=cap)
    return [i for i, w in enumerate(weights) if rule.check(w)]
```

Sources

- dbutils.secrets.get(scope, key) reads a secret and values are redacted in notebook output. (verified: https://docs.databricks.com/aws/en/security/secrets/ (checked 2026-09-25))
- In DAX, ALL(table) inside CALCULATE removes all filters on that table; ALL on one dimension keeps other filters. (verified: Taught and tested in lesson C6-L2; DAX CALCULATE and ALL semantics per Microsoft Learn DAX reference.)

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D5-L1 Invented functions, parameters and syntax

D5-L1 exercise 1

Hints. (1) Is delta() a DataFrameReader method? (2) Check with hasattr(spark.read, 'delta'). (3) No.

Answer. a) AttributeError: DataFrameReader has no attribute 'delta' Because: r1) The reader has format('delta').load(path) and table(name), but no delta() method.

Plausible, and invented. `hasattr` or the PySpark API reference answers in seconds.

Common wrong answers.

- `b`: The method does not exist. Use spark.read.format("delta").load(path) or spark.read.table(name).
- `c`: No read happens. The attribute lookup fails first.
- `d`: Python raises AttributeError at once.
- `a|r2`: Right answer, wrong reason. Reading by path works with format('delta').load(path).

D5-L1 exercise 2

Hints. (1) What does Column do with an unknown attribute? (2) It assumes a struct field. (3) The result is another Column.

Answer.

```
1 Column
```

`F.col("region").isnotnull` means "the field isnotnull inside the struct region". Calling it raises `TypeError: 'Column' object is not callable`.

Common wrong answers.

- `method`: The real method is isNotNull, with capitals. isnotnull is treated as a struct field name.
- `AttributeError`: Column turns unknown attributes into field lookups, so no error appears here.

D5-L1 exercise 3

Hints. (1) Which statement comes from another platform? (2) Databricks binds masks with ALTER TABLE. (3) ALTER COLUMN ... SET MASK.

Answer. The bug is on line 3.

Fixed version:

```
1 CREATE FUNCTION research_governance.mask_postcode(p STRING)
2   RETURN CASE WHEN is_account_group_member('analysts') THEN p ELSE '***'
    END;
3 ALTER TABLE research_silver.nat_respondents ALTER COLUMN postcode SET
    MASK research_governance.mask_postcode;
```

Databricks column masks: a SQL UDF, then `ALTER TABLE ... ALTER COLUMN ... SET MASK`.

Common wrong answers.

- Line 1: A SQL UDF is exactly what Databricks column masks use.
- Line 2: is_account_group_member is the recommended check in Unity Catalog.

D5-L1 exercise 4

Hints. (1) `inspect.signature(func).parameters` maps names to Parameter objects. (2) `p.kind is inspect.Parameter.VAR_KEYWORD` marks **kwargs. (3) Keep the order of params.

Answer.

```
1  import inspect
2  from collections.abc import Callable
3
4
5  def unknown_params(func: Callable[..., object], params: list[str]) ->
     list[str] | None:
6      """Parameter names that a function does not accept.
7
8      Args:
9          func: Function to check.
10         params: Names suggested for it, for example by an assistant.
11
12     Returns:
13         Unknown names in order, or None if func takes **kwargs.
14     """
15     parameters = inspect.signature(func).parameters
16     if any(p.kind is inspect.Parameter.VAR_KEYWORD for p in
     parameters.values()):
17         return None
18     return [name for name in params if name not in parameters]
```

A signature check catches an invented parameter before the code runs. With **kwargs, only the docs can answer.

D5-L1 exercise 5

Hints. (1) What proves an API exists for your version? (2) Which reference covers Databricks SQL? (3) What does 'undocumented' guarantee?

Answer.

- A docs link for the right version turns "looks right" into "checked".
- SQL dialects share keywords and differ in statements. Only the platform reference counts.
- Undocumented behaviour can change or disappear without notice.

Docs for your version, the platform's own SQL reference, and no undocumented APIs.

Common wrong answers.

- check = `trust`: Assistants state invented APIs with the same confidence as real ones.
- sql = `generic`: CREATE ROW ACCESS POLICY appears in tutorials. It is Snowflake, not Databricks.
- unverified = `try`: Running is necessary, not sufficient. Undocumented behaviour has no guarantee.

D5-L2 Deprecated APIs and version mismatches

D5-L2 exercise 1

Hints. (1) Check the 17.3 LTS release notes. (2) File details live in a hidden metadata column. (3) _metadata.file_path.

Answer. a) The _metadata.file_path column Because: r1) Databricks release notes state input_file_name is no longer supported from 17.3 LTS. The file metadata column replaces it.

Select `F.col("_metadata.file_path")` when reading files.

Common wrong answers.

- `b`: It is no longer supported on 17.3 LTS and above.
- `c`: os.path works on strings, not on DataFrame columns.
- `d`: A UDF cannot see the source file. The metadata column can.
- `a|r2`: Right answer, wrong reason. It was not renamed. A different mechanism, the _metadata column, replaces it.

D5-L2 exercise 2

Hints. (1) What does pandas 3 do with chained assignment? (2) The intermediate object behaves as a copy. (3) The original keeps 9.0.

Answer.

```
[1.0, 9.0]
```

pandas 3 warns with ChainedAssignmentError and leaves df unchanged. Use `df.loc[1, "w"] = 5.0`.

Common wrong answers.

- `[1.0, 5.0]`: That was pandas 1.x behaviour. Under Copy-on-Write, df['w'] behaves as a copy, so the original is unchanged.

D5-L2 exercise 3

Hints. (1) DataFrame.append is gone. What joins frames end to end? (2) applymap was renamed. (3) `concat` and `map`.

Answer.

```
combined = pd.concat([waves, new_wave], ignore_index=True)
labels = combined.map(str.strip)
```

`pd.concat` replaces `append`. `DataFrame.map` replaces `applymap`.

Common wrong answers.

- g2 = `applymap`: applymap was removed in pandas 3.0. DataFrame.map replaces it.

D5-L2 exercise 4

Hints. (1) Split on dots and read the leading digits of each part. (2) Compare tuples of ints. (3) Pad the shorter tuple with zeros.

Answer.

```
import re

SEGMENT = re.compile(r"^(\d+)")


def _parse(version: str) -> tuple[int, ...]:
    """Numeric segments of a dotted version string."""
    parts = []
    for segment in version.split("."):
        match = SEGMENT.match(segment)
        if match is None:
            raise ValueError(f"invalid version segment {segment!r} in
    {version!r}")
        parts.append(int(match.group(1)))
```

```
    return tuple(parts)


def version_at_least(installed: str, required: str) -> bool:
    """True if installed is the same as or newer than required.

    Args:
        installed: Installed version, such as "3.0.2".
        required: Minimum version, such as "3.0".

    Returns:
        Whether installed >= required, compared numerically.

    Raises:
        ValueError: If a version has a segment without digits.
    """
    a, b = _parse(installed), _parse(required)
    width = max(len(a), len(b))
    return a + (0,) * (width - len(a)) >= b + (0,) * (width - len(b))
```

Tuples of integers compare element by element, which is version order. In real projects, use `packaging.version.Version`.

D5-L2 exercise 5

Hints. (1) Two indexing steps in one assignment. (2) Chained assignment. (3) out.loc[mask, 'final_weight'] = cap.

Answer. The bug is on line 6.

Fixed version:

```
import pandas as pd

def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
    """Return a copy with final_weight capped at cap."""
    out = df.copy()
    out.loc[out["final_weight"] > cap, "final_weight"] = cap
    return out
```

Index once with `.loc[rows, column]`. Chained indexing sets values on a temporary copy.

Common wrong answers.

- Line 5: The copy is right: it keeps the caller's frame unchanged.

D5-L3 Five review checks for AI-written code

D5-L3 exercise 1

Hints. (1) Where does the expected value come from? (2) From the same formula. (3) Self-confirming.

Answer. a) It repeats the implementation, so a wrong formula passes Because: r1) A test's expected value must come from outside the code: small data worked by hand.

Write `assert weighted_pct(25, 200) == 12.5` and `assert weighted_pct(1, 0) is None`: numbers you worked out yourself.

Common wrong answers.

- `b`: approx helps with floats, but the test still repeats the formula.
- `c`: Arithmetic is fine. Copying the implementation is the problem.
- `d`: It checks that the code equals itself.
- `a|r2`: Right answer, wrong reason. Tests call the function. The expected value must be independent.

D5-L3 exercise 2

Hints. (1) Which line would leak through Git history? (2) A literal credential. (3) Read it at run time from the scope.

Answer. The bug is on line 3.

Fixed version:

```
1 import requests
2
3 def fetch_wave(wave_id: str) -> dict[str, object]:
4     """Fetch wave metadata from the survey platform."""
5     token = dbutils.secrets.get("survey-api", "token")
6     headers = {"Authorization": f"Bearer {token}"}
7     return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Secrets come from a secret scope at run time. Databricks also redacts values read with `dbutils.secrets.get` in notebook output.

Common wrong answers.

- Line 7: The header format is right. The value it uses comes from a literal on line 3.
- Line 8: The request is fine, with a timeout even. The secret is the problem.

D5-L3 exercise 3

Hints. (1) What does ALL on a whole table remove? (2) Every filter on it. (3) Including the wave.

Answer. a) Weighted reach for all waves, all regions and all brands Because: r1) ALL(fact_reach) removes every filter on the fact table, including the wave slicer.

Name the dimension whose filter you mean to remove: `ALL(dim_brand)` keeps the wave and region filters.

Common wrong answers.

- `b`: That is what ALL(dim_brand) would give. ALL on the fact table removes the wave filter too.
- `c`: That is the numerator's context.
- `d`: ALL accepts a table. That is exactly the problem here.
- `a|r2`: Right answer, wrong reason. CALCULATE keeps slicer filters unless a modifier such as ALL removes them.

D5-L3 exercise 4

Hints. (1) Check lengths before zip: zip stops at the shorter list. (2) Filter pairs with `is not None`. (3) Guard the total before dividing.

Answer.

```python
def weighted_mean(values: list[float | None], weights: list[float | None]) -> float | None:
    """Weighted mean that skips missing pairs.

    Args:
        values: Values, None for missing.
        weights: Weights, None for missing.

    Returns:
        The weighted mean, or None if there is nothing to average.

    Raises:
        ValueError: If lengths differ or a weight is negative.
    """
    if len(values) != len(weights):
        raise ValueError("values and weights differ in length")
    pairs = [(v, w) for v, w in zip(values, weights) if v is not None and w is not None]
    if any(w < 0 for _, w in pairs):
        raise ValueError("weights must not be negative")
    total = sum(w for _, w in pairs)
    if total == 0:
        return None
```

```
    return sum(v * w for v, w in pairs) / total
```

Five edge cases the one-liner missed: different lengths, None values, negative weights, empty input and a zero total.

D5-L3 exercise 5

Hints. (1) What does the code do, stripped of structure? (2) One comparison per weight. (3) A list comprehension with enumerate.

Answer.

```
def flag_high_weights(weights: list[float], cap: float) -> list[int]:
    """Indexes of weights strictly above cap.

    Args:
        weights: Respondent weights.
        cap: Largest allowed weight.

    Returns:
        Indexes of weights greater than cap.
    """
    return [i for i, w in enumerate(weights) if w > cap]
```

Three classes and a factory wrapped one comparison. Add structure when a second rule actually arrives.